

3. ANÁLISE DE COMPLEXIDADE PESSIMISTA

- O processo de construção de um algoritmo seria mais eficaz se a complexidade fosse um fator integrante do **projeto** do mesmo.
- Apesar de a **complexidade** ser uma **medida muito particular** à classe do algoritmo que está sendo analisado, alguns aspectos do cálculo da complexidade não dependem do que faz o algoritmo.
- Este capítulo introduz uma metodologia para analisar a **complexidade pessimista** (i. e. no pior caso) de um algoritmo com base em sua estrutura.
- Entre as medidas de complexidade, a **complexidade no pior caso** é o critério de avaliação mais utilizado.

- Serão estudadas, uma a uma, as principais *estruturas algorítmicas* a seguir:
 - *Atribuição*: $\mathbf{v} \leftarrow \mathbf{e}$,
 - *Seqüência* (ou composição): $\mathbf{S} ; \mathbf{T}$,
 - *Condicional*: se $\mathbf{b}$ então $\mathbf{S}$ senão $\mathbf{T}$ (ou se $\mathbf{b}$ então $\mathbf{S}$),
 - *Iteração definida* (ou incondicional): para $\mathbf{i}$ de $\mathbf{j}$ até $\mathbf{m}$ faça $\mathbf{S}$,
 - *Iteração indefinida* (ou condicional): enqto $\mathbf{b}$ faça $\mathbf{S}$.
- Na seção 2 do capítulo 2 (cf. 2.2.1), introduzimos as seguintes variantes para o critério pessimista de complexidade de um algoritmo $\mathbf{a}$:

$$c_{\overline{\mathbf{p}}}[\mathbf{a}](n) := \text{Máx} \{ \text{desemp}[\mathbf{a}](d) \in \text{IN} / \text{tam}(d) = n \},$$

$$c_{\mathbf{p}}^{\leq}[\mathbf{a}](n) := \text{Máx} \{ \text{desemp}[\mathbf{a}](d) \in \text{IN} / \text{tam}(d) \leq n \};$$
 onde $\text{desemp}[\mathbf{a}](d) := \text{custo}(\text{exec}[\mathbf{a}](d))$.
- Neste capítulo daremos ênfase à variante $c_{\mathbf{p}}^{\leq}[\mathbf{a}](n)$, entretanto vários resultados podem ser adaptados para a variante $c_{\overline{\mathbf{p}}}[\mathbf{a}](n)$.

Visão geral do capítulo

- 3.1 Preliminares e conceitos auxiliares.

Introduz algumas idéias básicas importantes e conceitos auxiliares, como os de absorção e máximo assintótico em ordem.

- 3.2 Equações de complexidade pessimista.

Estuda a complexidade de cada uma das principais estruturas algorítmicas, estabelecendo equações, ilustradas por exemplos simples.

- 3.3 Exemplos de aplicação.

Ilustra a metodologia da seção 3.2, aplicando-a a alguns exemplos de algoritmos.

3.1 PRELIMINARES E CONCEITOS AUXILIARES

- A motivação é reduzir o desempenho - e a complexidade - de um algoritmo a informações sobre suas partes.
- Consideremos funções **f** e **g** de um conjunto **D** no conjunto $\mathbb{R}_+$. Definimos:
soma pontual por: $(f + g)(d) := f(d) + g(d)$, para cada $d \in D$.
- O **máximo e mínimo pontuais**, respectivamente, por:
 $\text{Máx}(f, g)(d) := \text{Máx}\{f(d), g(d)\}$ e $\text{mín}(f, g)(d) := \text{mín}\{f(d), g(d)\}$.

3.1.1 Idéias básicas: componentes de algoritmos

- A complexidade de um algoritmo poderá ser obtida a partir das complexidades das suas partes.
- Em geral, cada **componente executada** dá sua contribuição para o desempenho e a complexidade do algoritmo.
- Torna-se conveniente classificar as componentes de um algoritmo em **conjuntivas** ou **disjuntivas**, conforme sejam sempre executadas ou não.

- Por exemplo, no (trecho de) algoritmo
 $j \leftarrow i + 1 ; k \leftarrow i - 1 ;$
 essas duas atribuições são suas componentes conjuntivas.
- Por outro lado, no (trecho de) algoritmo
 $\text{se } i \neq j \text{ então } j \leftarrow i + 1 \text{ senão } k \leftarrow i - 1 ;$
 essas duas atribuições são suas componentes disjuntivas.
- Consideremos um algoritmo $\mathbf{a}_0$ com duas componentes $\mathbf{a}_1$ e $\mathbf{a}_2$.
- Para uma entrada $\mathbf{d}$ com $\text{tam}(\mathbf{d}) \leq n$, sabemos que:
 $\text{desemp}[\mathbf{a}_j](\mathbf{d}) \leq c_P^\leq [\mathbf{a}_j](n)$, para $j = 0, 1, 2$.
- Para o caso de **componentes conjuntivas**:
 $\text{desemp}[\mathbf{a}_0](\mathbf{d}) = \text{desemp}[\mathbf{a}_1](\mathbf{d}) + \text{desemp}[\mathbf{a}_2](\mathbf{d}) \quad (3.1.1).$
- Como as complexidades pessimistas dão cotas superiores para os desempenhos, temos:
 $\text{desemp}[\mathbf{a}_0](\mathbf{d}) \leq c_P^\leq [\mathbf{a}_1](n) + c_P^\leq [\mathbf{a}_2](n).$

- Assim, pela definição de **complexidade pessimista**, obtemos a seguinte **cota superior**:

$$c_P^{\leq} [a_0](n) \leq c_P^{\leq} [a_1](n) + c_P^{\leq} [a_2](n).$$

- Por outro lado, pela equação (3.1.1) os desempenhos das partes dão cotas inferiores:

$$desemp[a_j](d) \leq desemp[a_0](d), \text{ para cada } j = 1, 2.$$

- De novo, como a complexidade pessimista é cota superior para o desempenho, temos:

$$desemp[a_i](d) \leq c_P^{\leq} [a_0](n), \text{ para cada } i = 1, 2.$$

- Assim, pela definição de **complexidade pessimista**, obtemos as seguintes **cotas inferiores**:

$$c_P^{\leq} [a_j](n) \leq c_P^{\leq} [a_0](n), \text{ para cada } j = 1, 2.$$

- Portanto, da definição de **máximo pontual**, vem a seguinte **cota inferior** para $c_P^{\leq} [a_0]$:

$$\text{Máx} (c_P^{\leq} [a_1], c_P^{\leq} [a_2])(n) \leq c_P^{\leq} [a_0](n).$$

- Considere um algoritmo $\mathbf{a}_0$, com duas componentes conjuntivas $\mathbf{a}_1$ e $\mathbf{a}_2$ ($desemp[\mathbf{a}_0](d) = desemp[\mathbf{a}_1](d) + desemp[\mathbf{a}_2](d)$, para qualquer entrada $\mathbf{d}$).

Princípio das partes conjuntivas

- Então a **complexidade pessimista** do algoritmo $\mathbf{a}_0$ tem cotas:

$$\text{Máx} (c_P^{\leq} [\mathbf{a}_1], c_P^{\leq} [\mathbf{a}_2]) \leq c_P^{\leq} [\mathbf{a}_0] \leq c_P^{\leq} [\mathbf{a}_1] + c_P^{\leq} [\mathbf{a}_2].$$

- Logo, a complexidade pessimista do algoritmo $\mathbf{a}_0$ tem ordem $\Theta(c_P^{\leq} [\mathbf{a}_1] + c_P^{\leq} [\mathbf{a}_2])$.

- Consideremos agora um algoritmo $\mathbf{a}_0$ com duas componentes disjuntivas $\mathbf{a}_1$ e $\mathbf{a}_2$.
- Neste caso, temos esforços computacionais associados às execuções ora da componente $\mathbf{a}_1$, ora da componente $\mathbf{a}_2$.
- Assim, para cada entrada $\mathbf{d}$, temos:

$$desemp[\mathbf{a}_0](\mathbf{d}) = desemp[\mathbf{a}_1](\mathbf{d}) \text{ ou } desemp[\mathbf{a}_0](\mathbf{d}) = desemp[\mathbf{a}_2](\mathbf{d}) \quad (3.1.2).$$
- Como as complexidades pessimistas dão cotas superiores para os **desempenhos**, temos:

$$desemp[\mathbf{a}_0](\mathbf{d}) \leq c_P^{\leq} [\mathbf{a}_1](\mathbf{n}) \text{ ou } desemp[\mathbf{a}_0](\mathbf{d}) \leq c_P^{\leq} [\mathbf{a}_2](\mathbf{n}).$$
- Assim, pela definição de **máximo pontual**, obtemos a seguinte **cota superior**:

$$desemp[\mathbf{a}_0](\mathbf{d}) \leq \text{Máx} (c_P^{\leq} [\mathbf{a}_1], c_P^{\leq} [\mathbf{a}_2])(\mathbf{n}).$$
- Portanto, pela definição de **complexidade pessimista**, obtemos a seguinte **cota superior**: $c_P^{\leq} [\mathbf{a}_0](\mathbf{n}) \leq \text{Máx} (c_P^{\leq} [\mathbf{a}_1], c_P^{\leq} [\mathbf{a}_2])(\mathbf{n}).$

- Por outro lado, como a complexidade pessimista é cota superior para o desempenho, da expressão (3.1.2) temos:
 se $desemp[a_0](d) = desemp[a_1](d)$, então: $desemp[a_1](d) \leq c_P^{\leq} [a_0](n)$;
 se $desemp[a_0](d) = desemp[a_2](d)$, então: $desemp[a_2](d) \leq c_P^{\leq} [a_0](n)$.
- Assim, para qualquer entrada d com $tam(d) \leq n$, temos a seguinte cota inferior:

$$\text{Máx} \{ desemp[a_1](d), desemp[a_2](d) \} \leq c_P^{\leq} [a_0](n).$$
- Logo, para o **pior caso** de entradas com tamanho até n , temos a **cota inferior**:

$$\text{Máx} \{ \text{Máx} \{ desemp[a_1](d), desemp[a_2](d) \} / tam(d) \leq n \} \leq c_P^{\leq} [a_0](n).$$
- Mas, pode-se mostrar que $\text{Máx} \{ c_P^{\leq} [a_1](n), c_P^{\leq} [a_2](n) \}$ é uma cota inferior para:

$$\text{Máx} \{ \text{Máx} \{ desemp[a_1](d), desemp[a_1](d) \} / tam(d) \leq n \}.$$

- Portanto, a definição de **máximo pontual**, vem a ser **cota inferior** para $c_P^{\leq} [a_0]$:

$$\text{Máx} (c_P^{\leq} [a_1], c_P^{\leq} [a_2])(n) \leq c_P^{\leq} [a_0](n).$$
- Considere um algoritmo a_0 , cujas duas componentes a_1 e a_2 são disjuntivas, de modo que $(\text{desemp}[a_0](d) = \text{desemp}[a_1](d) + \text{desemp}[a_2](d))$, para qualquer entrada d).

Princípio das partes disjuntivas

- A **complexidade pessimista** do algoritmo a_0 tem cotas:

$$c_P^{\leq} [a_0] = \text{Máx} (c_P^{\leq} [a_1], c_P^{\leq} [a_2]).$$

- Logo, a complexidade pessimista das componentes a_1 e a_2 têm ordens $\Omega(c_P^{\leq} [a_0])$.

3.2.2 Conceitos auxiliares

- Os conceitos de absorção e máximo assintótico (em ordem) são motivados por idéias de reduzir a complexidade de um algoritmo às ordens das suas partes.
- A idéia de uma componente de um algoritmo **absorver outra** se aplica quando ambas as componentes são **conjuntivas**. Para o caso de componentes **disjuntivas**, aparece a idéia de **máximo assintótico**, que será considerada em seguida.
- Uma complexidade é **absorvida** por outra, por exemplo, se ambas são polinômios na mesma variável. Nesse caso, a complexidade de menor grau é absorvida. Assim: n é absorvida por n^2 ; também $n \cdot \log n$ é absorvida por n^2 .

- Dadas funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, dizemos que **f** é absorvida por **g**, sse, **f** é $O(g)$. Isto significa que:
 $(\exists c \in \mathbb{R}_+) (\exists n_0 \in \mathbb{N}) (\forall n \geq n_0): f(n) \leq c \cdot g(n)$.

Princípio da absorção

- Para funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, se **f** é absorvida por **g**, sua **soma pontual**:
f + g é $\Theta(g)$.
- Assim, como $n \cdot \log n$ é absorvida por n^2 , a soma **($n \cdot \log n + n^2$)** é $\Theta(n^2)$.

Máximo assintótico

- Considerando funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, vamos a seguir examinar algumas expressões mais simples para a ordem do **máximo pontual** $\text{Máx} (f, g)(n) := \text{Máx} \{ f(n), g(n) \}$.
- Frequentemente, a partir de certo ponto, $f(n)$ **fica sempre maior** do que $g(n)$, ou vice-versa.
Mais geralmente: se **f** é $O(g)$, então $\text{Máx} (f, g)$ também é $O(g)$.
Exemplo: como $2n + 3$ é $O(n^2)$, então $\text{Máx} (2n + 3, n^2)$ é $O(n^2)$.
- Pode acontecer, no entanto, que **não exista uma dominância**. Quando isso acontece, uma solução simplista, mas muitas vezes usada, é utilizar para a ordem do máximo pontual $\text{Máx} (f, g)$ a **soma pontual**, dada por $(f + g)(n) := f(n) + g(n)$.
- Consideremos, por sua vez, funções **f** e **g** de $\mathbb{N}$ em $\mathbb{N}$. O que devemos esperar de seu **máximo assintótico** $\text{MxAO} (f, g)$?

- Inicialmente, é claro que o máximo assintótico deve ser **comutativo**.

(i): $\text{MxAO} (f, g) = \text{MxAO} (g, f)$.

- Devemos também requerer que as funções sejam **dominadas**.

(ii): $f = O(\text{MxAO} (f, g))$ e $g = O(\text{MxAO} (f, g))$.

- Para ter uma cota menos folgada podemos exigir que o máximo assintótico seja a melhor **cota superior**.

(iii): $\text{MxAO} (f, g) = O(h)$, sempre que $f = O(h)$ e $g = O(h)$.

- Para tratar melhor o caso de dominância de uma das funções podemos requerer também a seguinte propriedade.

(iv): $\text{MxAO} (f, g) = g$, sempre que $(\exists n_0 \in \mathbb{N}) (\forall n \geq n_0): f(n) \leq g(n)$.

3.2 EQUAÇÕES DE COMPLEXIDADE PESSIMISTA

- Precisamos das complexidades das componentes básicas e saber como combinar complexidades de componentes.
- Vamos analisar a complexidade de cada uma das principais **estruturas algorítmicas**.
- Daremos ênfase à variante $c_p^{\leq} [a]$, cuja notação simplificaremos para $c_p [a]$. Entretanto, vários resultados podem ser adaptados para a variante $c_p^{\equiv} [a]$.

3.2.1 Atribuição

Exemplo 3.2.1. Considere as atribuições a seguir:

- a) Para variáveis inteiras i e j :
- | | |
|------------------|------------------|
| $i \leftarrow 0$ | {inicialização} |
| $j \leftarrow i$ | {transferência}. |

b) Para lista v de inteiros e variável inteira m :

$m \leftarrow \text{Max}(v)$ {valor máximo}.

Esta atribuição envolve (sendo n o comprimento da lista em v):

- determinar o máximo da lista v , com complexidade $\Theta(n)$;
- transferir este valor, com complexidade $\Theta(1)$.

c) Para listas u , v e w :

$u \leftarrow v$ {transfere lista}

$w \leftarrow \text{Reversa}(v)$ {inverte lista}.

A atribuição de transferência de cada elemento da lista v , para uma lista u com comprimento n , tem complexidade $O(n)$.

A atribuição $w \leftarrow \text{Reversa}(v)$ envolve:

- inverter a lista v , com complexidade $\Theta(n)$;
- transferir os elemento da lista invertida, com complexidade $\Theta(n)$.

- Assim, o desempenho da atribuição $v \leftarrow e$, com entrada d é dado por:

$$desemp[v \leftarrow e](d) = \text{cálcl}[e](d) + \text{transf}[e(d)].$$

- Dada uma atribuição $v \leftarrow e$, sejam $c_P [e]$ a complexidade de avaliação da expressão e e $c_P [\leftarrow_e]$ a complexidade de transferência do valor resultante.

Complexidade pessimista da atribuição

- A complexidade pessimista da atribuição $v \leftarrow e$ tem cotas:

$$\text{Máx} (c_P [e], c_P [\leftarrow_e]) \leq c_P [v \leftarrow e] \leq c_P [e] + c_P [e].$$

- Logo, a ordem **da complexidade pessimista** da atribuição $v \leftarrow e$ é dada por:

$$c_P [v \leftarrow e] = O(c_P [e] + c_P [e]).$$

Exemplo 3.2.2. Para listas u e v de inteiros, temos:

$$c_P [u \leftarrow \text{Ordene}(v)] = O(c_P [\text{Ordene}] + c_P [\leftarrow_{\text{Ordene}(v)}]).$$

- Supondo que Ordene é um algoritmo com **tempo** $O(n^2)$, teremos:

$$c_P [u \leftarrow \text{Ordene}(v)] = O(n^2 + n) = O(n^2).$$

- Note que, se a operação fundamental é a **comparação**, então:

$$c_P [u \leftarrow \text{Ordene}(v)] = O(c_P [\text{Ordene}]).$$

3.2.2 Seqüência (ou composição)

Exemplo 3.2.3. Considere as seqüências a seguir:

- a) Para variáveis inteiras i e j :

$$i \leftarrow 0 ; j \leftarrow i.$$

b) Para lista v de inteiros e variável inteira m :

$m \leftarrow \text{Max}(v)$; $m \leftarrow m + 1$.

c) Para listas u , v e w :

$u \leftarrow v$; $w \leftarrow \text{Reversa}(v)$.

- Assim, o desempenho da seqüência **S ; T** com entrada **d** é dado por:

$$\text{desemp}[S ; T](d) = \text{desemp}[S](d) + \text{desemp}[T](S(d)).$$

- Note que a execução de **S** pode **alterar o tamanho dos dados**. Isto pode ocorrer, por exemplo, no caso em que **S** e **T** atuam sobre uma lista, a execução de **S** reduz o tamanho da lista e **T** faz uma procura seqüencial na lista.

Complexidade da seqüência com preservação (assintótica) de tamanho

- Se **S** preserva tamanho assintoticamente, então a complexidade pessimista da seqüência **S ; T** tem cotas:

$$\text{Máx} (c_P [S], c_P [T]) \leq c_P [S ; T] \leq c_P [S] + c_P [T].$$

- A ordem da **complexidade pessimista** da seqüência **S ; T** é dada por:

$$c_P [S ; T] = O(c_P [S] + c_P [T]).$$

Exemplo 3.2.4. Considere o trecho de algoritmo:

$v \leftarrow \text{Reversa}(u) ;$

$w \leftarrow \text{Ordene}(v).$

Reversa e Ordene têm complexidades $\Theta(n)$ e $O(n^2)$, respectivamente.

- Complexidade pessimista da seqüência (em geral). Um exemplo em que não há preservação de tamanho:

Exemplo 3.2.5. Dados algoritmos $\text{Prim}(u)$ e $\text{Buscab}(a,v)$, considere a seqüência:

$$v \leftarrow \text{Prim}(u) ; \text{Buscab}(a, v).$$

Vamos supor que:

- $\text{Prim}(u)$ dá como saída a primeira metade da lista em u , com comprimento $\lfloor n / 2 \rfloor$, e tem complexidade $\Theta(n)$;
- $\text{Buscab}(a, v)$ procura a na lista v , com complexidade $O(\log m)$, para lista v com comprimento m .

- Dada uma seqüência **S ; T**, considere $s(n)$ dando o tamanho máximo da saída **S** para entradas com tamanho até n , i. e.:

$$s(n) := \text{Máx} \{ \text{tam}(S(d)) / \text{tam}(d) \leq n \}.$$

Complexidade pessimista da seqüência (em geral)

- A complexidade pessimista da seqüência **S ; T** tem cotas:

$$\text{Máx} (c_P [S](n), c_P [T](s(n))) \leq c_P [S ; T](n) \leq c_P [S](n) + c_P [T](s(n)).$$

- Logo, a **complexidade pessimista** da seqüência **S ; T** é dada por:

$$c_P [S ; T](n) = O(c_P [S](n) + c_P [T](s(n))).$$

Exemplo 3.2.6. Considere a seqüência:

$v \leftarrow \text{Prim}(u) ; w \leftarrow \text{Fin}(u) ;$
 $v \leftarrow \text{Ordene}(v) ; w \leftarrow \text{Ordene}(w) ;$
 $u \leftarrow \text{Concat}(v , w).$

Vamos supor que:

- Prim é como no exemplo 3.2.5 e, analogamente, Fin dá a segunda metade de sua entrada;
- Ordene tem complexidade quadrática;
- Concat(v , w) dá como saída a concatenação das listas v e w , com complexidade $O(p + q)$.

3.2.3 Condicionais

se b então S ou se b então S senão T

Exemplo 3.2.7. Considere as estruturas condicionais a seguir:

a) Para variável inteira i:

se $i = 0$ então $i \leftarrow i + 1$.

Esta estrutura condicional envolve:

- determinar se o valor de i é 0, com complexidade $\Theta(1)$;
- se sim, executar a atribuição $i \leftarrow i + 1$, com complexidade $\Theta(1)$.

b) Para lista v de inteiros e variável inteira m:

se $m = 0$ então $m \leftarrow \text{Max}(v)$.

Esta atribuição envolve:

- determinar se o valor de m é 0, com complexidade $\Theta(1)$;
- se sim, executar a atribuição $m \leftarrow \text{Max}(v)$, com complexidade $\Theta(n)$.

- O desempenho $desemp[\text{se } b \text{ então } S](d)$ da estrutura condicional se **b** então **S**,

com entrada **d** é dado por:

$$\begin{array}{ll} aval [b](d) + desemp[S](d) & \text{caso o valor de } b \text{ em } d \text{ seja verdadeiro;} \\ aval [b](d) & \text{caso o valor de } b \text{ em } d \text{ seja falso.} \end{array}$$

Complexidade da estrutura condicional simples

- A complexidade pessimista de se **b** então **S** tem cotas:

$$c_P [b] \leq c_P [\text{se } b \text{ então } S] \leq c_P [b] + c_P [S].$$

- Logo, a ordem da **complexidade pessimista** de se **b** então **S** é dada por:

$$c_P [\text{se } b \text{ então } S] = O(c_P [b] + c_P [S]).$$

Exemplo 3.2.8. Dados algoritmos Max e Ordene, (ex. 3.2.1 e 3.2.2), considere o seguinte trecho de algoritmo, para lista v de inteiros:

se $\text{Max}(v) = 0$ então $v \leftarrow \text{Ordene}(v)$.

Para esta estrutura condicional temos:

$$\begin{aligned} c_P [\text{se } \text{Max}(v) = 0 \text{ então } v \leftarrow \text{Ordene}(v)] &= \\ &= O(c_P [\text{Max}(v) = 0] + c_P [v \leftarrow \text{Ordene}(v)]). \end{aligned}$$

Como seus componentes têm complexidades pessimistas:

$$c_P [\text{Max}(v) = 0] = O(n) \text{ e } c_P [v \leftarrow \text{Ordene}(v)] = O(n^2);$$

Exemplo 3.2.9. Considere as estruturas condicionais a seguir:

a) Para variáveis inteiras i e j :

se $i \neq j$ então $i \leftarrow i + j$ senão $j \leftarrow i + 1$.

Esta estrutura condicional envolve:

- saber se os valores de i e j são diferentes, tem complexidade $\Theta(1)$;
- se sim, executar a atribuição $i \leftarrow i + j$, com complexidade $\Theta(1)$;
- se não, executar a atribuição $j \leftarrow i + 1$, com complexidade $\Theta(1)$.

b) Para listas u e v (de inteiros):

se $u = v$ então $v \leftarrow \text{Prim}(u)$ senão $u \leftarrow \text{Ordene}(v)$.

Esta estrutura condicional envolve:

- determinar se as listas u e v são iguais, com complexidade $O(n)$;
- se sim, executar a atribuição $v \leftarrow \text{Prim}(u)$, com complexidade $O(n)$;
- se não, executar a atribuição $u \leftarrow \text{Ordene}(v)$, com complexidade $O(n^2)$.

- Assim, a estrutura condicional se **b** então **S** senão **T**, com entrada **d** tem desempenho $desemp[\text{se } b \text{ então } S \text{ senão } T](d)$ dado por:

$aval [b](d) + desemp[S](d)$ caso o valor de **b** em **d** seja verdadeiro;
 $aval [b](d) + desemp[T](d)$ caso o valor de **b** em **d** seja falso.

Complexidade pessimista da estrutura condicional composta

- A complexidade pessimista de se **b** então **S** senão **T** tem as cotas:

inferior : $Máx (c_P [b], c_P [S], c_P [T]) \leq c_P [\text{se } b \text{ então } S \text{ senão } T]$;

superior : $c_P [\text{se } b \text{ então } S \text{ senão } T] \leq c_P [b] + MxAO (c_P [S], c_P [T])$.

- A ordem da **complexidade pessimista** de se **b** então **S** senão **T** é dada por:

$$c_P [\text{se } b \text{ então } S \text{ senão } T] = O(c_P [b] + MxAO (c_P [S], c_P [T])).$$

Exemplo 3.2.10. Dados algoritmos Max, Reversa e Ordene (ex. 3.2.1 e 3.2.2), considere o seguinte trecho de algoritmo, para lista v de inteiros:

se $\text{Max}(v) \geq 0$ então $v \leftarrow \text{Reversa}(v)$ senão $v \leftarrow \text{Ordene}(v)$.

Para esta estrutura condicional temos:

$$\begin{aligned} c_p [\text{se } \text{Max}(v) \geq 0 \text{ então } v \leftarrow \text{Reversa}(v) \text{ senão } v \leftarrow \text{Ordene}(v)] &= \\ &= O(c_p [\text{Max}(v) \geq 0] + \text{MxAO}(c_p [v \leftarrow \text{Reversa}(v)], c_p [v \leftarrow \text{Ordene}(v)])). \end{aligned}$$

Suas componentes têm as seguintes complexidades pessimistas:

$$c_p [\text{Max}(v) \geq 0] = O(n);$$

$$c_p [v \leftarrow \text{Reversa}(v)] = O(n) \text{ e } c_p [v \leftarrow \text{Ordene}(v)] = O(n^2).$$

- Assim, temos o seguinte máximo assintótico em ordem de complexidades:

$$\begin{aligned} \text{MxAO}(c_p [v \leftarrow \text{Reversa}(v)], c_p [v \leftarrow \text{Ordene}(v)]) &= \\ &= \text{MxAO}(n, n^2) = O(n^2). \end{aligned}$$

3.2.4 Iteração definida (ou incondicional)

para i de j até m faça S

Exemplo 3.2.11. Considere as iterações definidas a seguir:

a) Para variável inteira m:

para i de 1 até 20 faça $m \leftarrow m + 1$.

b) Para vetor $A[1..n]$ de inteiros:

para i de 1 até n faça $A[i] \leftarrow 0$ {inicialização}.

c) Para vetor $A[1..n]$ de inteiros:

para i de 1 até n faça $A[i] \leftarrow A[i] + 1$ {atualização}.

d) Para variável real s e vetor $R[1..n]$ de reais

para i de 1 até n faça $s \leftarrow s + R[i]$ {somatório}.

e) Para variável real t e matriz quadrada $Q[1..n, 1..n]$ de reais

para i de 1 até n faça $t \leftarrow t + Q[i, i]$ {traço}.

- Na execução, para a entrada **d**, do comando:

para **i** de **j** até **m** faça **S**,

- inicialmente, **S** é executado, com $i = j$, sobre a entrada **d** dando $S(d)$;
- em seguida, **S** é executado, com $i := j + 1$, sobre $S(d)$ dando $S^2(d)$;
-
- finalmente, **S** é executado, com $i := m$, sobre $S^{N(d)-1}(d)$ resultando $S^{N(d)}(d)$

$N(d) = m - j + 1$, com **j** = **j**(d) e **m** = **m**(d).

- Além do esforço de controle dos valores de i , j , m , temos os **esforços computacionais** associados às várias iterações de **S** sucessivamente sobre:
 $d, S(d), S^2(d), \dots, S^{N(d)-1}(d)$.
- Em geral, os esforços computacionais de determinação dos valores de controle são dominados pelos das iterações de **S**.
- Assim, a iteração definida para i de j até m faça **S**, com entrada **d** tem seu desempenho $desemp[\text{para } i \text{ de } j \text{ até } m \text{ faça } S](d)$ dado pelo somatório:

$$\sum_{i=j(d)}^{m(d)} desemp[S](S^{(i-j(d))}(d)).$$

Exemplo 3.2.12. Considere a iteração definida:

para i de 1 até $(n - 1)$ faça Troca($A[i]$, $A[i + 1]$);

onde Troca($A[p]$, $A[q]$) troca de posição os elementos $A[p]$ e $A[q]$ no vetor $A[1..n]$, com complexidade constante $O(1)$.

São executadas $(n - 1)$ trocas, sucessivamente:

Troca($A[1]$, $A[2]$), Troca($A[2]$, $A[3]$), ..., Troca($A[n - 1]$, $A[n]$);
 cada uma delas com mesma complexidade $O(1)$.

- Dada uma iteração definida

para i de j até m faça **S**,

considere $N(n) := \text{Máx} \{N(d) \mid \text{tam}(d) \leq n\}$, onde $N(d) := (\mathbf{m}(d) - \mathbf{j}(d) + 1)$.

Iteração definida com preservação (assintótica) de tamanho

- Se **S** preserva tamanho assintoticamente, então a complexidade pessimista da iteração definida para i de j até m faça **S** tem cotas:

$$c_P [S](n) \leq c_P [\text{para } i \text{ de } j \text{ até } m \text{ faça } S](n) \leq N(n) \cdot c_P [S](n).$$

- Assim, a ordem da **complexidade pessimista** de para i de j até m faça **S** é:

$$c_P [\text{para } i \text{ de } j \text{ até } m \text{ faça } S](n) = O(N(n) \cdot c_P [S](n)).$$

Exemplo 3.2.13. Considere o trecho de algoritmo:

para i de 1 até p faça Buscab(A[i],v);

onde Buscab(a, v) é como no ex. 3.2.5: procura a na lista v, com complexidade $O(\log m)$, para lista v com comprimento m.

Considere como tamanho do problema o máximo entre p e m, chamando $n := \text{Máx}\{p, m\}$.

Exemplo 3.2.14. Considere a iteração definida

para i de j até m faça **S** onde:

S preserva tamanho, i. e., $\text{tam}(S(d)) = \text{tam}(d)$; e

$c_p[S]$ é um polinômio de grau k, i. e., $c_p[S](n) = O(n^k)$.

- Na execução da iteração definida

para i de j até m faça S,

em geral, **S** **pode não preservar o tamanho** (nem mesmo assintoticamente). Neste caso, cada iteração tem que ser considerada e a iteração da função tamanho será usada.

- Dada uma função $s : \mathbb{N} \rightarrow \mathbb{N}$, define-se por indução sobre $i \in \mathbb{N}$:

$$\begin{aligned} s^0(n) &:= n \\ s^{(i+1)}(n) &:= s^i(s(n)). \end{aligned}$$

A execução de **S** é iterada $N(d)$ vezes, onde $N(d) = (m(d) - j(d) + 1)$.

- Usando os valores máximo para $m(d)$ e mínimo para $j(d)$, para entrada de tamanho n :

$$m(n) := \text{Máx} \{ m(d) / tam(d) \leq n \} \text{ e } j(n) := \text{mín} \{ j(d) / tam(d) \leq n \}$$

e função tamanho máximo é

$$s(n) := \text{Máx} \{ tam(S(d)) / tam(d) \leq n \}.$$

Complexidade pessimista da iteração definida (em geral)

- A complexidade pessimista da iteração definida para i de j até m faça **S** tem as cotas:

inferior : Máx { $c_P [S](s^{(i-j(n))}(n))$ / $i = j(n), \dots, m(n)$ };

superior : $\sum_{i=j(n)}^{m(n)} c_P [S](s^{(i-j(n))}(n)).$

- Assim, a ordem da **complexidade pessimista** de para i de j até m faça **S** é:

$$[\text{para } i \text{ de } j \text{ até } m \text{ faça } S](n) = O\left(\sum_{i=j(n)}^{m(n)} c_P [S](s^{(i-j(n))}(n))\right).$$

Exemplo 3.2.15. Considere o trecho de algoritmo:

para i de 1 até n faça Buscas(A[i], B[i..n]);

onde Buscas(a, B[i..n]) é similar ao ex. 3.2.13: procura a na parte [i..n] (de i até n) do vetor B, com complexidade $O(m)$ (onde $m := n - i + 1$).

A iteração definida tem complexidade pessimista:

$$\begin{aligned} c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça Buscas(A[i], B[i..n]) }] &= \\ &= O\left(\sum_{i=1}^n c_P [\text{Buscas(A[i], B[i..n]) }](s^{(i-1)}(n))\right). \end{aligned}$$

Neste caso, temos:

$$c_P [\text{Buscas(A[i], B[i..n]) }](m) = O(m);$$

$$s(m) = m - 1 \text{ (dando } s^k(m) = m - k \text{)}.$$

$$\text{Assim, } c_P [\text{Buscas(A[i], B[i..n]) }](s^{(i-1)}(n)) = O(n - i + 1).$$

Então, complexidade pessimista da iteração definida fica:

$$c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça } \text{Buscas}(A[i], B[i..n])] = O(\sum_{i=1}^n (n - i + 1)).$$

Mas, $\sum_{i=1}^n (n - i + 1) = \sum_{j=1}^n j$ e sabemos que:

$$\sum_{j=1}^n j = \frac{(1+n) \cdot n}{2} = O(n^2).$$

Portanto, a ordem da **complexidade pessimista** desta iteração definida é:

$$\begin{aligned} c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça } \text{Buscas}(A[i], B[i..n])] &= \\ &= O(\sum_{j=1}^n j) = \mathbf{O(n^2)}. \end{aligned}$$

Exemplo 3.2.16. Considere a iteração definida

para i de 1 até n faça **S**

onde:

$c_P [S]$ é um polinômio de grau k , i. e., $c_P [S](n) = O(n^k)$; e $tam(S(d)) = tam(d) - 1$ (portanto $s(n) = n - 1$ e $s^i(n) = n - i$).

Neste caso, temos:

$$c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça } S](n) = O\left(\sum_{i=1}^n (n - i)^k\right).$$

Pode-se mostrar que $\sum_{i=1}^n (n - i)^k = O(n^{k+1})$.

Portanto, teremos:

$$c_P [\text{para } i \text{ de } 1 \text{ até } n \text{ faça } S](n) = O(n^{k+1}).$$

3.2.5 Iteração indefinida (ou condicional)

enqto **b** faça **S**

Exemplo 3.2.17. Considere os trechos de algoritmos a seguir:

a) Para variável inteira i:

$i \leftarrow 0$ {inicialização} ;
enqto $i \leq 10$ faça $i \leftarrow i + 1$ {iteração}.

b) Inicialização de vetor $A[1..n]$ de inteiros:

$i \leftarrow 0$ {inicialização} ;
enqto $i < n$ faça $i \leftarrow i + 1$; $A[i] \leftarrow 0$ fim-enqto {iteração}.

c) Atualização de vetor $A[1..n]$ de inteiros:

$i \leftarrow n$ {inicialização} ;
enqto $i > 0$ faça $A[i] \leftarrow A[i] + 1$; $i \leftarrow i - 1$ fim-enqto {iteração}.

d) Somatório de vetor $R[1..n]$ de reais:

$s \leftarrow 0$; $i \leftarrow n$ {inicializações} ;
enqto $i > 0$ faça $s \leftarrow s + R[i]$; $i \leftarrow i - 1$ fim-enqto {iteração}.

e) Traço de matriz quadrada $Q[1..n, 1..n]$ de reais:

$t \leftarrow 0$; $i \leftarrow 1$ {inicializações} ;
enqto $i \leq n$ faça $t \leftarrow t + Q[i, i]$; $i \leftarrow i + 1$ fim-enqto {iteração}.

f) Fatorial de natural n :

$f \leftarrow 1$; $i \leftarrow n$ {inicializações} ;
enqto $i > 1$ faça $f \leftarrow f \cdot i$; $i \leftarrow i - 1$ fim-enqto {iteração}.

A execução da iteração indefinida:

enqto **b** faça **S**,

sobre a entrada **d**, itera a execução de **S** sucessivamente:

- se a entrada **d** **satisfaz b**, então se executa **S** sobre **d**, dando **S(d)**;
- se **S(d)** **satisfaz** a condição **b**, então se executa **S** sobre **S(d)**, dando **S²(d)**;

.....

- finalmente, quando a condição **b deixa de ser satisfeita** sobre **S^{H(d)}(d)**, a execução termina com **S^{H(d)}(d)** como sua saída final.

Esforço computacional:

- avaliação de **b** e execução de **S** para: **d**, **S(d)**, **S²(d)**, ..., **S^{H(d)-1}(d)**;
- avaliação de **b** para **S^{H(d)}**.

- Uma questão **importante** é a determinação de **H(d)**.

- O desempenho de enqto **b** faça **S** sobre entrada **d** é dado por:

$$\begin{aligned} \text{desemp}[\text{enqto } b \text{ faça } S](d) = & \text{aval}[b](S^{H(d)}(d)) + \\ & + \sum_{i=0}^{H(d)-1} [\text{aval}[b](S^i(d)) + \text{desemp}[S](S^i(d))]. \end{aligned}$$

- Nos casos em que os testes são dominados pelos das iterações de **S**, podemos desprezá-los. Assim, a expressão anterior se simplifica para:

$$\text{desemp}[\text{enqto } b \text{ faça } S](d) = \sum_{i=0}^{H(d)-1} \text{desemp}[S](S^i(d)).$$

Exemplo 3.2.18. Considere a seguinte iteração indefinida:

enqto ($A[i] = 0 \wedge 1 \leq i \leq n$) faça $A[i] \leftarrow A[i] + 1$; $i \leftarrow i + 1$ fim-enqto.

Aqui, cada iteração mantém fixo o tamanho do vetor $A[1..n]$ de entrada e tem complexidade $O(1)$.

No pior caso, tem-se inicialmente $i = 1$ e são executados todas as iterações, com $i = 1, i = 2, \dots, i = n$;

Assim, essa **iteração indefinida** terá complexidade $n \cdot 1 = O(n)$.

- Agora é preciso **determinar $H(d)$** .

$H(d)$ é o número de vezes que **S** precisa ser executado para que o teste **b** dê o valor falso:

$$H(d) := \text{mín} \{ i \in \mathbb{N} / \neg b(S^i(d)) \}.$$

Como estamos interessados no **pior caso**, introduzimos:

$$h(n) := \text{Máx} \{ H(d) \in \mathbb{N} / \text{tam}(d) \leq n \}.$$

- Em casos razoavelmente freqüentes, a complexidade de **S** domina a do teste **b**, **absorvendo-a**.
- Supondo que $c_P[\mathbf{b}]$ é sempre dominada por $c_P[\mathbf{S}]$:

A definição da função $h : \mathbb{N} \rightarrow \mathbb{N}$ segue a idéia usada para $s : \mathbb{N} \rightarrow \mathbb{N}$, onde

$$s(n) := \text{Máx} \{ \text{tam}(S(d)) \in \mathbb{N} / \text{tam}(d) \leq n \}.$$

Complexidade da iteração indefinida com testes absorvidos

- A complexidade pessimista de enqto **b** faça **S** tem as cotas:

inferior : $\text{Máx} \{ c_P [S](s^i(n)) / i = 0, \dots, h(n) - 1 \} \leq c_P [\text{enqto } b \text{ faça } S](n);$

superior : $c_P [\text{enqto } b \text{ faça } S](n) \leq \sum_{i=0}^{h(n)-1} c_P [S](s^i(n)).$

- Se $c_P [b]$ é $O(c_P [S])$, a **complexidade pessimista** de enqto **b** faça **S** é:

$$c_P [\text{enqto } b \text{ faça } S](n) = O\left(\sum_{i=0}^{h(n)-1} c_P [S](s^i(n)) \right).$$

Exemplo 3.2.19. Considere a seguinte iteração indefinida:

enqto $1 \leq i \leq n$ faça Buscas($A[i]$, $B[i..n]$); $i \leftarrow i + 1$ fim-enqto,
onde Buscas(a , $B[i..n]$) é como no ex. 3.2.15: procura a na parte $[i..n]$ (de i até n) do vetor B , com complexidade $O(m)$.

Desprezando o teste, a iteração indefinida tem complexidade pessimista:

$$c_P [\text{enqto } 1 \leq i \leq n \text{ faça Buscas}(A[i], B[1..i]); i \leftarrow i + 1 \text{ fim-enqto}] = \\ = O\left(\sum_{i=0}^{h(n)-1} c_P [\text{Buscas}(A[i], B[1..i]); i \leftarrow i + 1](s^i(n)) \right).$$

Neste caso, temos: $c_P [\text{Buscas}(A[i], B[1..i]); i \leftarrow i + 1](m) = O(m)$, $h(n) = n$ e $s(m) = m - 1$ (dando $s^i(m) = m - i$).

Logo $c_P [\text{Buscas}(A[i], B[1..i]); i \leftarrow i + 1](s^i(n)) = O(n - i)$.

Procedendo como no ex. 3.2.15, obtemos: $\sum_{i=0}^{n-1} (n - i) = O(n^2)$.

• Portanto, a ordem da **complexidade pessimista** desta iteração indefinida é

$$c_P [\text{enqto } 1 \leq i \leq n \text{ faça Buscas}(A[i], B[1..i]); i \leftarrow i + 1 \text{ fim-enqto}] = \\ = O\left(\sum_{i=0}^{n-1} (n - i) \right) = O(n^2).$$

Complexidade pessimista da iteração indefinida (em geral)

- A complexidade pessimista de enqto **b** faça **S** tem as cotas:

inferior : Máx { $c_P [b](s^{h(n)}(n))$, Máx { $\text{iter}_i(n) / i = 0, \dots, h(n) - 1$ } }
 (onde $\text{iter}_i(n) := \text{Máx} \{ c_P [b](s^i(n)), c_P [S](s^i(n)) \}$);

superior : $c_P [b](s^{h(n)}(n)) + \sum_{i=0}^{h(n)-1} [c_P [b](s^i(n)) + c_P [S](s^i(n))]$.

- Assim, a ordem da **complexidade pessimista** de enqto **b** faça **S** é dada por:

$$c_P [\text{enqto } b \text{ faça } S](n) = O(c_P [b](s^{h(n)}(n)) + \sum_{i=0}^{h(n)-1} [c_P [b](s^i(n)) + c_P [S](s^i(n))]).$$

Exemplo 3.2.20. Considere a iteração indefinida enqto **b** faça **S** onde:

$c_P [S]$ é um polinômio de grau k , i. e., $c_P [S](n) = O(n^k)$; e h é no máximo linear, i. e., $h(n) \leq n$.

a) Se **S** preserva tamanho, temos $s(n) = n$.

$$c_P [\text{enqto } b \text{ faça } S](n) = O\left(\sum_{i=0}^{h(n)-1} n^k \right) = O\left(\sum_{i=0}^{n-1} n^k \right).$$

b) Se **S** faz o tamanho decrescer de uma unidade, temos $s(n) = n - 1$.

$$c_P [\text{enqto } b \text{ faça } S] = O\left(\sum_{i=0}^{h(n)-1} (n - i)^k \right) = O\left(\sum_{i=0}^{n-1} (n - i)^k \right).$$

Ambos os casos têm a mesma ordem:

$$\sum_{i=0}^{n-1} n^k \text{ e } \sum_{i=0}^{n-1} (n - i)^k \text{ são } O(n^{k+1}).$$

3.3 EXEMPLOS DE APLICAÇÃO

- Algoritmo 3.3.1: **Máximo e mínimo de tabela.**

<u>Função</u> Max_&_min(Tab : D)	→ Par[Elmord]	<i>{para máximo e mínimo de tabela}</i>
		<i>{Entrada: tabela Tab : D.}</i>
		<i>{Saída: valores Max, min : Elmord (máximo e mínimo da entrada).}</i>
<u>sorte</u> D := <u>vetor</u> [p..q] <u>de</u> Elmord ;		
1. Max ← Tab[p] ;		<i>{candidato a máximo}</i>
2. min ← Tab[p] ;		<i>{candidato a mínimo}</i>
3. <u>para</u> i <u>de</u> p + 1 <u>até</u> q <u>faça</u>		<i>{varre tabela: de 3 a 6}</i>
4. <u>se</u> Tab[i] ← Max <u>então</u> Max ← Tab[i] ;		<i>{atualiza Max}</i>
5. <u>se</u> Tab[i] ← min <u>então</u> min ← Tab[i] ;		<i>{atualiza min}</i>
6. <u>fim-para</u> ;		<i>{3: i de p + 1 até q}</i>
7. <u>retorne-saída</u> (Max , min) ;		<i>{dá saída Max e min}</i>
8. <u>fim-Função</u> .		<i>{fim do algoritmo Max_&_min: máximo e mínimo de tabela}</i>

Escolhendo como operação fundamental a comparação entre elementos.

- Analisando:

$$c_P [\text{linha 1} ; \text{linha 2}] = O(1).$$

$$c_P [\text{linha 4}] + c_P [\text{linha 5}] = O(1).$$

$$\begin{aligned} c_P [\text{para linha 3 ... linha 6}] &= O(q - 1 - p + 1) \cdot c_P [\text{linha 4; linha 7}] = \\ &= O((n - 1) \cdot 1) = O(n). \end{aligned}$$

$$\begin{aligned} c_P [\text{Máx_ \& _mín}] &= O(c_P [\text{linha 1}] + c_P [\text{linha 2}] + c_P [\text{para linha 3 ... linha 6}]) \\ &= O(n + 0 + n) \\ &= O(n). \end{aligned}$$

○ **Algoritmo 3.3.2: União e interseção de conjuntos.**

<u>Função</u> Uni&Int(S, T : M) → Par[M]	<i>{para a união e interseção de conjuntos}</i>
	<i>{Entrada: conjuntos S, T : M (a operar).}</i>
	<i>Saída: conjuntos Uni, Int : M (Uni, união $S \cup T$, e Int, a interseção $S \cap T$).}</i>
<u>sorte</u> M := Conj[Elm] {conjuntos de elementos de Elm} ;	
<u>var</u> {auxiliar} x : Elm {elemento de Elm} ;	
1. Uni ← S ;	<i>{inicializa Uni com S}</i>
2. Int ← $\emptyset$;	<i>{inicializa Int com vazio}</i>
3. <u>para cada</u> x ∈ T <u>faça</u>	<i>{varre conjunto T: de 3 a 8}</i>
4. <u>se</u> x ∈ S	<i>{testa se x está em S}</i>
5. <u>então</u> Insere(x, Int)	<i>{põe x comum em Int}</i>
6. <u>senão</u> Insere(x, Uni) ;	<i>{põe novo x em Uni}</i>
7. <u>fim-se</u> ;	<i>{4: x ∈ S?}</i>
8. <u>fim-para</u> ;	<i>{3: cada x ∈ T}</i>
9. <u>retorne-saída</u> (Uni, Int) ;	<i>{dá saída Uni e Int}</i>
10. <u>fim-Função</u> .	<i>{fim de Uni&Int: união e interseção de conjuntos}</i>

- Analisando:

$$\begin{aligned}c_P [\text{Uni\&Int}] &= \\&= O(c_P [\text{Uni} \leftarrow S] + c_P [\text{Int} \leftarrow \emptyset] + c_P [\text{para linha 3 ... linha 8}]) \\&= O(|S| + 1 + (|T| \cdot c_P [\text{se linha 4 ... linha 7}])) \\&= O(n + 1 + (n \cdot (c_P [x \in S] + \text{MxAO} (c_P [\text{linha 5}], c_P [\text{linha 6}])))) \\&= O(n + (n \cdot (|S| + \text{MxAO} (1, 1)))) \\&= O(n + (n \cdot (n + 1))) \\&= O(n + n^2) \\&= O(n^2).\end{aligned}$$

○ **Algoritmo 3.3.3: Classificação de vetor.**

Procedimento Classif_Max(A : V)

{classifica vetor em ordem ascendente}

{Entrada: vetor A : V.

Saída: vetor A : V (o vetor de entrada classificado em ordem ascendente).}

sorte V := vetor [1..n] de Elmord ;

var {auxiliar} Temp : Elmord {temporária (para troca)} ;

1. para i de 1 até n - 1 faça

{de 1 a 10}

2. para j de 1 até n - i faça

{de 2 a 9}

3. se A[j] > A[j + 1]

{testa inversão}

4. então

{troca A[j] com A[j + 1]}

5. Temp ← A[j] ;

6. A[j] ← A[j + 1] ;

7. A[j + 1] ← Temp ;

8. fim-se ;

{3: A[j] ← A[j + 1]?}

9. fim-para ;

{2: j de 1 até n - i}

10. fim-para ;

{1: i de 1 até n}

11. retorne-saída(A) ;

{dá saída A}

12. fim-Procedimento .

{fim de Classif_Max: classificação ascendente}

○ **Algoritmo 3.3.4: Loteria esportiva.**

Procedimento Loto(Rslt : V, Apst : M) *{teste da loteria esportiva}*
{Entrada: vetor Rslt : V (resultado de teste), matriz Apst : M (apostador)
Saída: vetor Ptos : vetor [1..n] de IN (número de pontos feitos por apostador).}
 sorte V := vetor [1..13] de { 1..3 } ; M := matriz [1..n, 0..13] de Info ;

1. $i \leftarrow 1$; *{inicializa número de apostadores}*
2. enqto $i \leq n$ faça *{examina apostador: de 2 a 8}*
3. Ptos[i] $\leftarrow 0$; *{inicializa número de pontos do apostador i}*
4. para j de 1 até 13 faça *{examina palpites: de 4 a 6}*
5. se Apst[i, j] = Rslt[j] então Ptos[i] $\leftarrow$ Ptos[i] + 1 .
6. fim-para ; *{4: pontos do apostador i contabilizados}*
7. $i \leftarrow i + 1$; *{novo apostador}*
8. fim-enqto ; *{2: contabilizados os pontos de todos apostadores}*
9. para i de 1 até n faça *{lista ganhadores: de 9 a 11}*
10. se Ptos[i] = 13 então escreva(Apst[$i, 0$], "Ganhador, parabéns")
11. fim-para ; *{9: ganhadores listados}*
12. retorne-saída(Ptos) ; *{dá saída Ptos}*
13. fim-Procedimento . *{fim do algoritmo Loto: loteria esportiva}*

3.4 RESUMO DO CAPÍTULO

- A finalidade deste capítulo é introduzir uma metodologia para analisar a complexidade pessimista de um algoritmo com base em sua estrutura.
- Algumas componentes de um algoritmo são sempre executadas (as conjuntivas), outras (as disjuntivas) definem alternativas, que são executadas conforme o caso.
- As contribuições desses dois gêneros de componentes para o desempenho e a complexidade de um algoritmo **a**, com duas componentes **a₁** e **a₂** é dada por dois princípios básicos.
 - ▣ **Princípio das partes conjuntivas:** A complexidade pessimista de um algoritmo **a** com duas componentes **a₁** e **a₂** conjuntivas (de modo que $desemp[a](d) = desemp[a_1](d) + desemp[a_2](d)$, para qualquer entrada **d**) tem as cotas $Máx(c_P^{\leq}[a_1], c_P^{\leq}[a_2]) \leq c_P^{\leq}[a] \leq c_P^{\leq}[a_1] + c_P^{\leq}[a_2]$.

- ▣ **Princípio das partes disjuntivas:** A complexidade pessimista de um algoritmo **a** com duas componentes **a₁** e **a₂** disjuntivas (de modo que $desemp[a](d) = desemp[a_1](d)$ ou $desemp[a](d) = desemp[a_2](d)$, para qualquer entrada **d**) é dada por $c_p^\leq[a] = \text{Máx}(c_p^\leq[a_1], c_p^\leq[a_2])$.
- Os conceitos de absorção e máximo assintótico (em ordem) são motivados pelas idéias de reduzir a complexidade de um algoritmo às ordens das suas partes.
- A complexidade de uma parte pode ser "absorvida" pela complexidade de outra: a idéia é que a segunda domina a primeira, dando a contribuição principal para a soma. Para funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, dizemos que **f** é absorvida por **g** sse **f** é $O(g)$.
- O princípio seguinte formula a idéia acima: se a função **f** cresce mais rapidamente do que **g**, então sua soma crescerá como **f**, de maneira assintótica.
 - ▣ **Princípio da absorção:** Para funções **f** e **g** de $\mathbb{N}$ em $\mathbb{R}_+$, se **f** é absorvida por **g**, sua soma pontual **f + g** é $\Theta(g)$.

- O máximo assintótico em ordem MxAO (f, g), deve ser uma boa cota superior - em sentido assintótico - para as ordens de ambas. Frequentemente, toma-se como máximo assintótico a soma ou o máximo pontuais.
- Complexidades pessimistas das principais estruturas algorítmicas:

▣ **Atribuição $v \leftarrow e$:**

$$c_P [v \leftarrow e] = O(c_P [e] + c_P [\leftarrow_e]);$$

onde $c_P [e]$ e $c_P [\leftarrow_e]$ são as complexidades pessimistas da avaliação da expressão e e da transferência do valor resultante, respectivamente.

▣ **Seqüência $S ; T$:**

$$c_P [S ; T](n) = O(c_P [S](n) + c_P [T](s(n)));$$

onde $s(n)$ dá o tamanho máximo da saída de S para entradas com tamanho até n , i. e.

$$s(n) := \text{Máx} \{ \text{tam}(S(d)) / \text{tam}(d) \leq n \}.$$

□ **Condicional se b então S senão T:**

$c_P [\text{se } b \text{ então } \mathbf{S} \text{ senão } \mathbf{T}] = O(c_P [b] + \text{MxAO}(c_P [\mathbf{S}], c_P [\mathbf{T}]))$.
onde MxAO dá um máximo assintótico em ordem das complexidades das partes.

□ **Iteração definida para i de j até m faça S:**

$$c_P [\text{para } i \text{ de } j \text{ até } m \text{ faça } \mathbf{S}](n) = O(\sum_{i=j(n)}^{m(n)} c_P [\mathbf{S}](s^{(i-j(n))}(n)));$$

onde $m(n) := \text{Máx} \{ m(d) / \text{tam}(d) \leq n \}$, $j(n) := \text{mín} \{ j(d) / \text{tam}(d) \leq n \}$, e a função $s : \mathbb{N} \rightarrow \mathbb{N}$ é como no caso de seqüência.

□ **Iteração indefinida enqto b faça **S**:**

$$c_P [\text{enqto } \mathbf{b} \text{ faça } \mathbf{S}](n) = O(c_P [\mathbf{b}](s^{h(n)}(n)) + \sum_{i=0}^{h(n)-1} [c_P [\mathbf{b}](s^i(n)) + c_P [\mathbf{S}](s^i(n))]);$$

onde $s : \mathbb{N} \rightarrow \mathbb{N}$, e $h(n)$ dá o número máximo de iterações para entradas com tamanho até n , i. e.

$h(n) := \text{Máx} \{ H(d) \in \mathbb{N} / \text{tam}(d) \leq n \},$

com $H(d) := \text{mín} \{ i \in \mathbb{N} / \neg b(S^i(d)) \}.$